## PROGRAM:

```
#include <stdio.h>

int n;
int e[10][10], a[10][10], c[10][10];

void closure(int s)
{
   int i, j;

   c[s][s] = 1;

   for (i = 0; i < n; i++)
      if (e[s][i])
         c[s][i] = 1;

   for (i = 0; i < n; i++)
      if (c[s][i])
         for (j = 0; j < n; j++)
            if (e[i][j])
               c[s][j] = 1;
}

int main()
{
   int i, j, k;

   printf("Enter number of states: ");
   scanf("%d", &n);

   printf("Enter epsilon transition matrix:\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         scanf("%d", &e[i][j]);

   printf("Enter transition matrix for 'a':\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         scanf("%d", &a[i][j]);

   /* Find epsilon closure */
   for (i = 0; i < n; i++)
      closure(i);

   printf("\nEpsilon Closure:\n");
   for (i = 0; i < n; i++)
   {
      printf("E(q%d) = { ", i);
      for (j = 0; j < n; j++)
         if (c[i][j])
            printf("q%d ", j);
      printf("}\n");
   }

   /* Find new transitions */
   printf("\nNFA without epsilon:\n");

   for (i = 0; i < n; i++)
   {
```

```
        printf("q%d --a--> { ", i);

        for (j = 0; j < n; j++)
        {
            if (c[i][j])
            {
                for (k = 0; k < n; k++)
                {
                    if (a[j][k])
                    {
                        int x;
                        for (x = 0; x < n; x++)
                            if (c[k][x])
                                printf("q%d ", x);
                    }
                }
            }
        }

        printf("}\n");
    }

    return 0;
}
```

**OUTPUT:**

```
dell@dell-OptiPlex-380:~/sandhra$ cc first.c
dell@dell-OptiPlex-380:~/sandhra$ ./a.out
Enter number of productions: 5
Enter productions:
E=TA
A=+TA
A=#
T=i
T=(E)

FIRST:
FIRST(E) = { i( }
FIRST(A) = { + }
FIRST(T) = { i( }

FOLLOW:
FOLLOW(E) = { $) }
FOLLOW(A) = { $) }
FOLLOW(T) = { + }
dell@dell-OptiPlex-380:~/sandhra$
```

## PROGRAM:

```
#include <stdio.h>
int main()
{
   int n, m, i, j;
   int trans[10][10][10];
   int dfa[20][10];
   int states = 1;

   printf("Enter number of states: ");
   scanf("%d", &n);

   printf("Enter number of input symbols: ");
   scanf("%d", &m);

   printf("Enter NFA transition table:\n");
   printf("(Enter destination states as 0 or 1)\n");

   for (i = 0; i < n; i++)
   {
      for (j = 0; j < m; j++)
      {
         printf("q%d on symbol %d:\n", i, j);

         int k;
         for (k = 0; k < n; k++)
            scanf("%d", &trans[i][j][k]);
      }
   }

   /* Start state = q0 */
   for (i = 0; i < n; i++)
      dfa[0][i] = 0;

   dfa[0][0] = 1;

   printf("\nDFA Transition Table:\n");

   for (i = 0; i < states; i++)
   {
      for (j = 0; j < m; j++)
      {
         int result[10] = {0};
         int k, x;

         for (k = 0; k < n; k++)
         {
            if (dfa[i][k])
            {
               for (x = 0; x < n; x++)
               {
                  if (trans[k][j][x])
                     result[x] = 1;
               }
            }
         }

         printf("{ ");
```

```
        for (k = 0; k < n; k++)
           if (dfa[i][k])
              printf("q%d ", k);

        printf("} --%d--> { ", j);

        for (k = 0; k < n; k++)
           if (result[k])
              printf("q%d ", k);

        printf("}\n");

        /* Add new state */
        int found = 0;

        for (x = 0; x < states; x++)
        {
           int same = 1;

           for (k = 0; k < n; k++)
           {
              if (dfa[x][k] != result[k])
                 same = 0;
           }

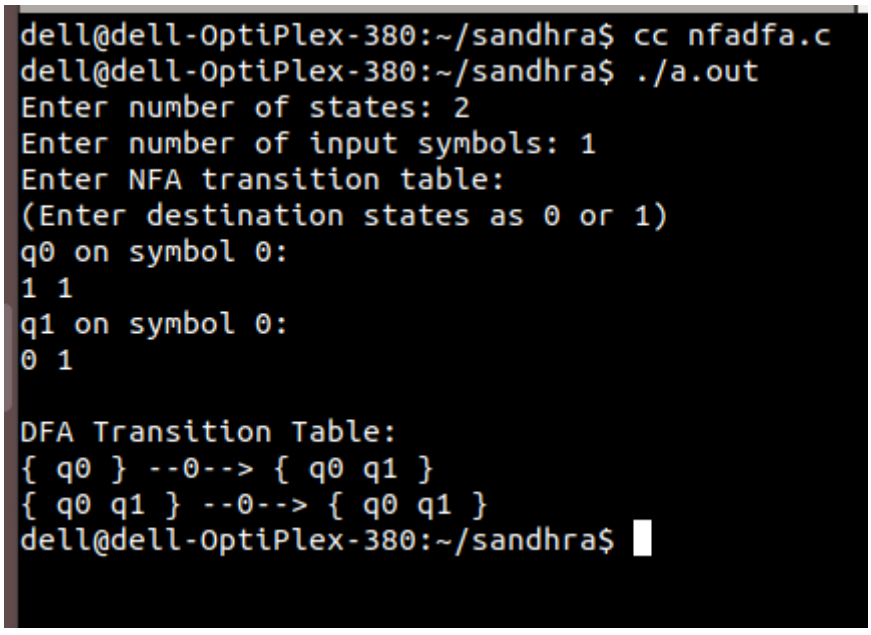
           if (same)
              found = 1;
        }

        if (!found)
        {
           for (k = 0; k < n; k++)
              dfa[states][k] = result[k];

           states++;
        }
     }
  }

  return 0;
}
```

**OUTPUT:**



## PROGRAM:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char p[10][20];
char first[26][20], follow[26][20];
int n;

void add(char s[], char c)
{
   int i;

   if (c == '#')
      return;

   for (i = 0; s[i]; i++)
      if (s[i] == c)
         return;

   s[i] = c;
   s[i + 1] = '\0';
}

void FIRST(char c)
{
   int i, j;
   char x;

   for (i = 0; i < n; i++)
   {
      if (p[i][0] == c)
      {
         x = p[i][2];

         if (!isupper(x))
            add(first[c - 'A'], x);
         else
         {
            FIRST(x);

            for (j = 0; first[x - 'A'][j]; j++)
               add(first[c - 'A'], first[x - 'A'][j]);
         }
      }
   }
}

void FOLLOW(char c)
{
   int i, j, k;

   /* $ for start symbol */
   if (c == p[0][0])
      add(follow[c - 'A'], '$');

   for (i = 0; i < n; i++)
   {
```

```
        for (j = 2; p[i][j]; j++)
        {
            if (p[i][j] == c)
            {
                if (p[i][j + 1] != '\0')
                {
                    char x = p[i][j + 1];

                    if (!isupper(x))
                        add(follow[c - 'A'], x);
                    else
                    {
                        for (k = 0; first[x - 'A'][k]; k++)
                            add(follow[c - 'A'],
                                first[x - 'A'][k]);
                    }
                }
                else if (p[i][0] != c)
                {
                    FOLLOW(p[i][0]);

                    for (k = 0; follow[p[i][0] - 'A'][k]; k++)
                        add(follow[c - 'A'],
                            follow[p[i][0] - 'A'][k]);
                }
            }
        }
    }
}

int main()
{
    int i;
    int printed[26] = {0};

    printf("Enter number of productions: ");
    scanf("%d", &n);

    printf("Enter productions:\n");

    for (i = 0; i < n; i++)
        scanf("%s", p[i]);

    /* FIRST */
    for (i = 0; i < n; i++)
        FIRST(p[i][0]);

    printf("\nFIRST:\n");

    for (i = 0; i < n; i++)
    {
        int x = p[i][0] - 'A';

        if (!printed[x])
        {
            printf("FIRST(%c) = { %s }\n",
                   p[i][0], first[x]);
            printed[x] = 1;
        }
    }
```

```
   /* FOLLOW */
   memset(printed, 0, sizeof(printed));

   for (i = 0; i < n; i++)
      FOLLOW(p[i][0]);

   printf("\nFOLLOW:\n");

   for (i = 0; i < n; i++)
   {
      int x = p[i][0] - 'A';

      if (!printed[x])
      {
         printf("FOLLOW(%c) = { %s }\n",
               p[i][0], follow[x]);
         printed[x] = 1;
      }
   }

   return 0;
}
```

**OUTPUT:**

